Activity 4 Report: Database Users, Roles, Privileges and Access Control

COMP 018 Database Administration | Project tested: HabalGo (Ride and Booking Management System) (PHP + PostgreSQL) | Date of testing: 7 November 2026

1. Cover and links

Item	Value
Testing group	G08: Borreta, Joshua; Bola, Martin Lawrence; Juatchon, Joaquin Phillip; Marcos, Karl Raphael; Pacheco Jr., Sancho; Vera Cruz, Euan Joseph
Project tested	HabalGo (Ride and Booking Management System)
Owning group	Group 6 (member names are not listed in the repository)
Repository	https://github.com/jeanaroyo/HabalGo
Tracker	https://github.com/EclipseChan/bwisIT-Solutions---Campus-Canteen-Pre-Order-and-Queue-Management-System/tree/main/other_group_tests
Evidence files	a4_access_tests.txt, a3_deploy.txt, a3_app.txt

This is a re-test. The database was dropped and rebuilt from the repository files, and every result below comes from this run.

2. Access-control plan (from the project files)

database\03_roles_and_privileges.sql, headed "Activity 4", defines two NOLOGIN group roles (app_runtime_grp for full read and write, app_readonly_grp for reading only), two login accounts (habalgo_app_user and habalgo_readonly_user), the grants on schema habalgo, and default privileges for future tables. The README does not mention roles or this script, and the application code has no role feature. The plan is database-level only.

3. The script, run as written

The script was run unedited on the freshly built database, and then a second time.

Check	Result
First run	Two errors: role "app_runtime_grp" and "app_readonly_grp" already exist. They were left on this server by another project tested earlier, so on this machine the group roles were reused and the rest of the script ran through (CREATE ROLE for the two logins, GRANT ROLE x2, GRANT x5, ALTER DEFAULT PRIVILEGES x3)
Second run	Four "already exists" errors (the groups and the two logins). The script is not repeatable. CREATE ROLE has no IF NOT EXISTS
Role attributes	Both logins: LOGIN only, not superuser, no CREATEDB, no CREATEROLE. Passwords stored as SCRAM-SHA-256. The passwords are written in plain text inside the script
Membership	habalgo_app_user is a member of app_runtime_grp,

	habalgo_readonly_user of app_readonly_grp (the script grants it, unlike some other projects)
Privilege matrix (S I U D T)	habalgo_app_user SIUD- on all 8 tables. habalgo_readonly_user S---- on all 8
Schema and database	USAGE on schema habalgo, no CREATE on habalgo or public, CONNECT on the database, USAGE on the sequences
Database ACL	{=Tc/postgres, postgres=CTc/postgres, app_runtime_grp=c/postgres, app_readonly_grp=c/postgres}
Default privileges	A new table created afterwards by postgres was readable and writable by app_runtime_grp and readable but not writable by app_readonly_grp (the test table was dropped again)
Other database	habalgo_app_user cannot connect to the postgres database (no CONNECT privilege)

4. Tests with real logins

Test	Expected	Actual
App user: read customers, insert a customer (rolled back)	Allowed	Allowed
App user: update and delete a payment (rolled back)	Allowed (full CRUD)	Allowed. Money records can be edited and deleted by the app account
App user: TRUNCATE booking	Denied	Denied (permission denied for table booking)
App user: CREATE TABLE in habalgo and in public	Denied	Denied (permission denied for schema)
App user: DROP, ALTER, CREATE INDEX	Denied	Denied (must be owner of table payment)
App user: CREATE ROLE, read pg_authid	Denied	Denied
App user: VACUUM	Denied (not owner)	Skipped with a warning
App user: CREATE TEMP TABLE	Not intended	Allowed. PUBLIC has TEMP on the database by default. Harmless here, a REVOKE would close it
Read-only user: select bookings	Allowed	Allowed
Read-only user: insert, update, delete	Denied	Denied (permission denied for table)
Read-only user: nextval on a sequence, CREATE TABLE	Denied	Denied

5. The application running as each account

As habalgo_app_user the web app worked for everything tested: dashboard, creating a customer, vehicle, booking, payment and rating, updating a booking status, deleting a vehicle and editing and deleting a payment. pg_stat_activity showed the connections belonging to that account. The one failing call was the driver assignment, which has the test-body gap described in Activity 3.

As habalgo_readonly_user the dashboard, customer list and booking list loaded. Creating a customer and deleting a customer both failed, and the API answered only "Database error." with HTTP 400, so a user of the screen would not be told it is a permission problem.

6. Who can do what at the API

The source files were searched for session, login, password_verify, token and Authorization and nothing was found. Every call in Activity 3 and this activity carried no cookie, token or password. Whoever can reach port 8000 can create, edit and delete customers, bookings and payments with the database account the server was

started with. That account is whatever DB_USER is set to. config.php defaults to postgres, and the README and .env.example both say postgres, so the default deployment runs the whole web app as a superuser.

pg_hba.conf uses scram-sha-256 on every rule (local, 127.0.0.1 and ::1), with no trust entries.

7. Issues and recommendations

#	Finding	Recommendation
1	No authentication or authorisation in the application. Anyone with network access can read and change everything, including payments	Add a login and roles in the app. At minimum, do not expose the server beyond localhost
2	The README and .env.example tell the user to run the app as postgres, and the least-privilege script is not mentioned. config.php also defaults to postgres	Point the README to habalgo_app_user and make it the default in .env.example
3	03_roles_and_privileges.sql fails on a server that already has roles with the same names and cannot be run twice. Passwords are fixed text in the file	Use DO blocks that check pg_roles first, and read the passwords from a variable
4	The application account has full write access to payment. A bug or a stolen password can change or erase payment history	Revoke UPDATE and DELETE on payment (and rating_feedback) from the runtime role, and add voided or refunded status rows instead
5	PUBLIC can create temporary tables, and the script does not revoke CREATE on the database	REVOKE TEMP ON DATABASE ... FROM PUBLIC if this is not wanted
6	The read-only role works at the database, but the app shows "Database error." for a permission failure	Map SQLSTATE 42501 to a clear message
7	No audit trail of who changed bookings or payments	Add a log table or trigger

8. Feedback for the owners

The role script itself does what it says and it is a good least-privilege layout, including membership and default privileges. The useful changes are to make it repeatable and to point the README at it.

9. Limitations

The groups app_runtime_grp and app_readonly_grp, and also app_admin_grp, already existed on this server from other projects, so their start state was not clean. Their privileges on this database come only from the HabalGo script (the database ACL and table grants above). The two login accounts were new. The driver assignment call was not exercised with a full body.

AI usage declaration

- Tool: Claude (Anthropic).
- Tasks: interpreting the results of the testing.
- Validation: the commands were run by the group on its own computer, and every result in this report comes from the saved output files.